

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical
```

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

x_train = x_train[:1000]
y_train = y_train[:1000]
x_test = x_test[:1000]
y_test = y_test[:1000]

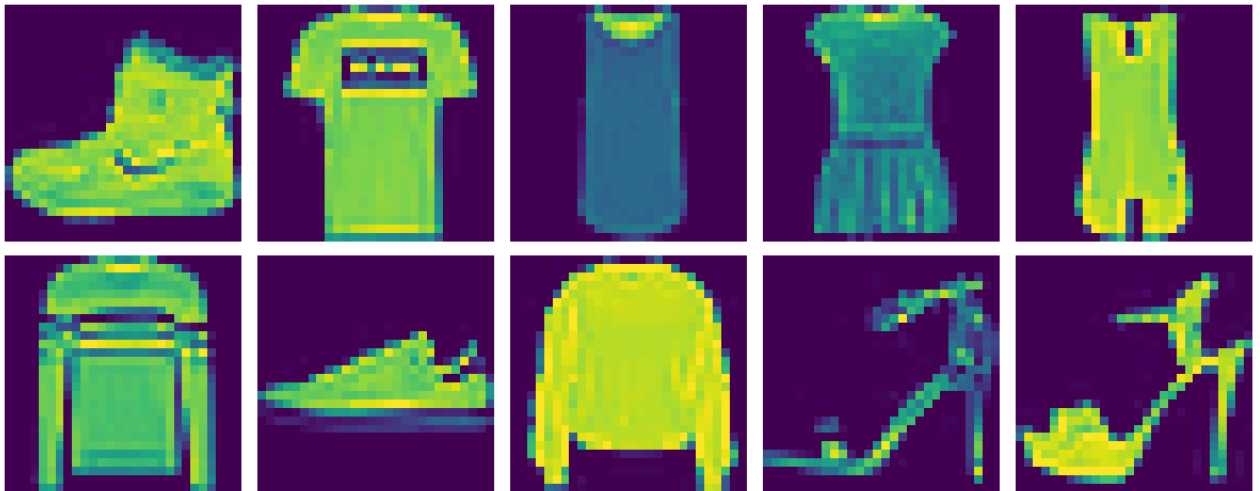
print("Training Images :", x_train.shape)
print("Training Labels :", y_train.shape)
print("Testing Images  :", x_test.shape)
print("Testing Labels  :", y_test.shape)
```

```
Training Images : (1000, 28, 28)
Training Labels : (1000,)
Testing Images  : (1000, 28, 28)
Testing Labels  : (1000,)
```

```
plt.figure(figsize=(12,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_train[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255
y_train = to_categorical(y_train,10)
y_test = to_categorical(y_test,10)
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np

# Reshape the data to include the channel dimension (rank 4)
# From (60000, 28, 28) to (60000, 28, 28, 1)
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
```

```

        height_shift_range=0.2,
        horizontal_flip=True
    )

    # Now fit will work with the rank 4 input
    datagen.fit(x_train)

    print("Data reshaped to:", x_train.shape)
    print("Datagen fitted successfully.")

```

Data reshaped to: (1000, 28, 28, 1)
Datagen fitted successfully.

```

# Fix: Use (1, 28, 28, 1) because Fashion MNIST images are 28x28
sample = x_train[0].reshape(1, 28, 28, 1)

aug = datagen.flow(sample, batch_size=1)

plt.figure(figsize=(12, 3))

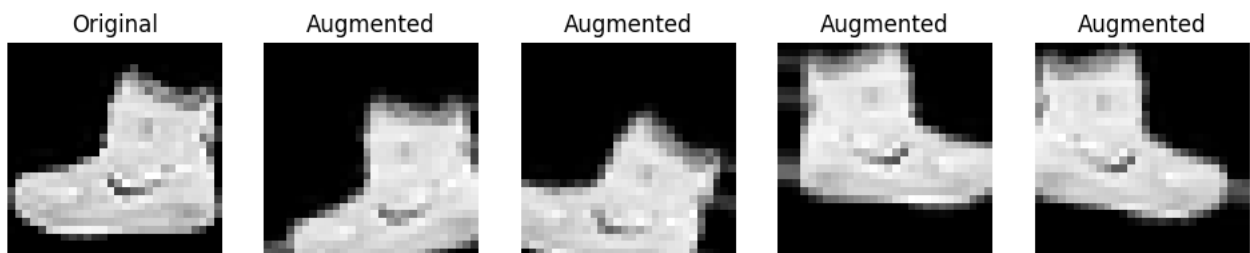
plt.subplot(1, 5, 1)
# Squeeze the (1, 28, 28, 1) to (28, 28) for matplotlib display
plt.imshow(sample[0].squeeze(), cmap='gray')
plt.title("Original")
plt.axis("off")

for i in range(4):
    image = next(aug)

    plt.subplot(1, 5, i + 2)
    # Squeeze the augmented image as well
    plt.imshow(image[0].squeeze(), cmap='gray')
    plt.title("Augmented")
    plt.axis("off")

plt.show()

```



```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    # Updated input_shape to (28, 28, 1) to match Fashion MNIST
    Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    MaxPooling2D(2, 2),

    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D(2, 2),

    Conv2D(128, (3, 3), activation="relu"),
    Flatten(),

    Dense(128, activation="relu"),
    Dropout(0.5),
    Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy", # Changed to categorical since y_train is one-hot encoded
    metrics=["accuracy"]
)

model.summary()

```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_6 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_10 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_11 (Conv2D)	(None, 3, 3, 128)	73,856
flatten_3 (Flatten)	(None, 1152)	0
dense_6 (Dense)	(None, 128)	147,584
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 10)	1,290

Total params: 241,546 (943.54 KB)

Trainable params: 241,546 (943.54 KB)

Non-trainable params: 0 (0.00 KB)

◆ Gemini

```
import numpy as np
from tensorflow.keras.utils import to_categorical

# 1. Ensure data is normalized and reshaped to (samples, 28, 28, 1)
# We check max() to avoid double-normalization if the cell is re-run
x_train_fixed = x_train.astype('float32') / 255.0 if x_train.max() > 1 else x_train
x_test_fixed = x_test.astype('float32') / 255.0 if x_test.max() > 1 else x_test

if len(x_train_fixed.shape) == 3:
    x_train_fixed = np.expand_dims(x_train_fixed, axis=-1)
    x_test_fixed = np.expand_dims(x_test_fixed, axis=-1)

# 2. Fix the label rank issue: Only encode if they aren't already one-hot (rank 2)
y_train_cat = y_train if len(y_train.shape) == 2 else to_categorical(y_train, 10)
y_test_cat = y_test if len(y_test.shape) == 2 else to_categorical(y_test, 10)

# 3. Re-compile to ensure the model matches the (28, 28, 1) input shape
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# 4. Train
history = model.fit(
    datagen.flow(
        x_train_fixed,
        y_train_cat,
        batch_size=64
    ),
    validation_data=(x_test_fixed, y_test_cat),
    epochs=10
)
```

```
Epoch 1/10
16/16 ————— 4s 112ms/step - accuracy: 0.1540 - loss: 2.2577 - val_accuracy: 0.3890 - val_loss: 2.0453
Epoch 2/10
16/16 ————— 1s 93ms/step - accuracy: 0.3000 - loss: 1.9316 - val_accuracy: 0.4470 - val_loss: 1.5262
Epoch 3/10
16/16 ————— 2s 98ms/step - accuracy: 0.3580 - loss: 1.6815 - val_accuracy: 0.5350 - val_loss: 1.3626
Epoch 4/10
16/16 ————— 3s 163ms/step - accuracy: 0.4080 - loss: 1.5381 - val_accuracy: 0.5380 - val_loss: 1.2605
Epoch 5/10
16/16 ————— 2s 111ms/step - accuracy: 0.4370 - loss: 1.4583 - val_accuracy: 0.5690 - val_loss: 1.2419
Epoch 6/10
16/16 ————— 1s 92ms/step - accuracy: 0.4930 - loss: 1.3705 - val_accuracy: 0.5180 - val_loss: 1.2012
Epoch 7/10
16/16 ————— 2s 93ms/step - accuracy: 0.5040 - loss: 1.3369 - val_accuracy: 0.6240 - val_loss: 1.0667
Epoch 8/10
16/16 ————— 2s 98ms/step - accuracy: 0.5360 - loss: 1.2843 - val_accuracy: 0.6370 - val_loss: 1.0409
Epoch 9/10
16/16 ————— 2s 95ms/step - accuracy: 0.5260 - loss: 1.2760 - val_accuracy: 0.6510 - val_loss: 1.0188
Epoch 10/10
16/16 ————— 1s 93ms/step - accuracy: 0.5570 - loss: 1.2109 - val_accuracy: 0.5860 - val_loss: 1.0371
```

```
loss, accuracy = model.evaluate(x_test,y_test)

print("Accuracy:",accuracy)
```

32/32 ————— 1s 15ms/step - accuracy: 0.5860 - loss: 1.0371
Accuracy: 0.5860000252723694

```
plt.figure(figsize=(8,5))

plt.plot(history.history["accuracy"],
         label="Training")

plt.plot(history.history["val_accuracy"],
         label="Validation")

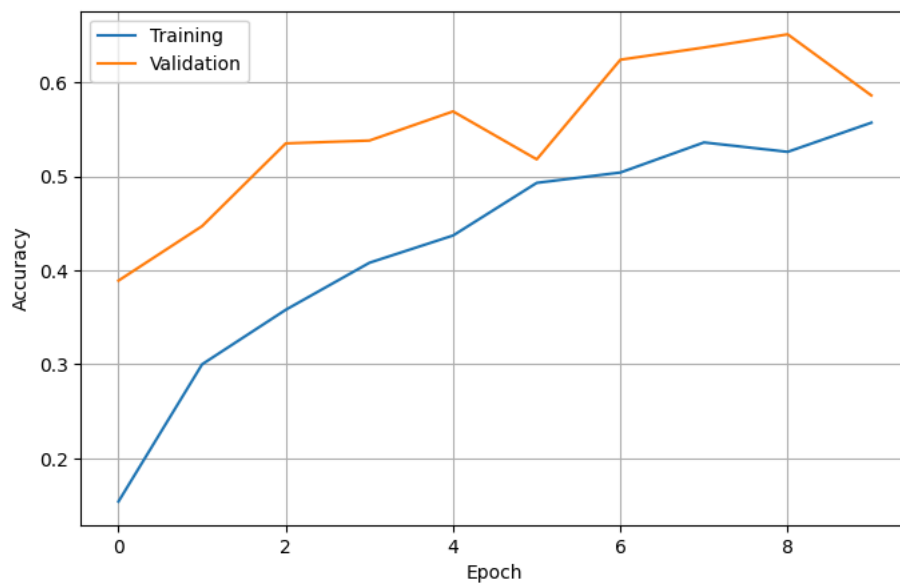
plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.grid()

plt.show()
```



Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.